



Webfejlesztési keretrendszerek

8. gyakorlat

Dr. Jánki Zoltán Richárd

Hol tartunk?

- UI elkészült
- Routing elkészült
- Adatok
 - az adatok az adatbázisban bent vannak
 - adatbázis műveletek még hiányosak
 - rétegelt architektúra ellenőrzése (service használat)



Service réteg szerepe

SZOFTVER ARCHITEKTÚRA ÉS AZ ÉTTEREM ANALÓGIA



Separation of Concerns (SoC)

HÁROMRÉTEGŰ SZOFTVERARCHITEKTÚRA

PRESENTATION / BUSINESS LOGIC / DATA ACCESS

1. MEGJELENÍTÉSI RÉTEG

(View / Komponens)



FELELŐSSÉG:

Kizárólag adatok megmutatása a felhasználónak; felhasználónak



Nem tudja az adatforrást, csak megjeleníti, amit kap.



Felhasználói Interakció → ← Megjelenített Adat



FELELŐSSÉG:

Kizárólag adatok megmutatása a felhasználónak; felhasználói interakciók összegyűjtése (kattintás, űrlap, görgetés).



Nem tudja az adatforrást, csak megjeleníti, amit kap.

2. ÜZLETI LOGIKA RÉTEG

(Business Logic / Service)



FELELŐSSÉG:

Adatok szűrése, átalakítása, validálása; backend hívások koordinálása; állapot kezelése.



Nem tud a DOM-ról/HTML-ről, csak adatokkal dolgozik.



Nyers adat

Feldolgozott adat



ADAT SZŰRÉS & ÁTALAKÍTÁS



VALIDÁLÁS & ÁLLAPOT KEZELÉS



BACKEND HÍVÁSOK KOORDINÁLÁSA

3. ADATELÉRÉSI RÉTEG

(Data Access / Backend)



FELELŐSSÉG:

Tényleges adatforrás. A service ezen keresztül éri el az adatokat, a konkrét implementáció (SDK, URL, protokoll) elrejtve.



REST API



Firestore



localStorage

REJTETT IMPLEMENTÁCIÓ
(SDK, URL)



Service réteg előnyei

- Újrafelhasználhatóság
 - több komponens is használhatja
- Tesztelhetőség
 - izoláltan tesztelhető, adatok mock-olhatóak
- Cserélhetőség
 - backend váltásnál csak a service réteg módosul
- Single Source of Truth
 - állapotkezelés egy helyen
- Közös hibakezelés
 - loading/error/retry logikát egy helyen kezeljük

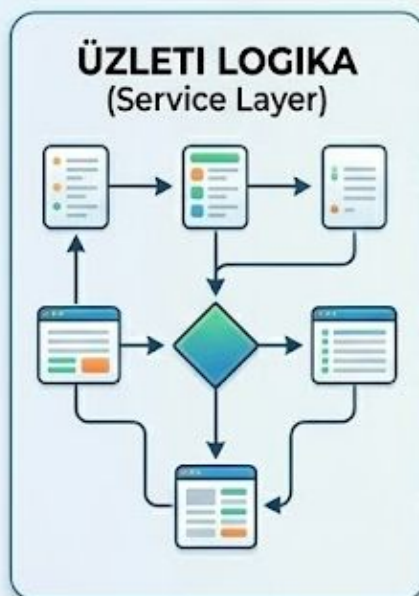


Polyglot perzisztencia

POLYGLOT PERZISZTENCIA A MODERN SZOFTVERARCHITEKTÚRÁBAN.

Szemléltetés és Jelentőség: Hogyan Működik és Miért Jó?

HOGYAN MŰKÖDIK?



ÁTMENET A MONOLITIKUS MODELLTŐL →



Monolitikus Adatbázis
(Jack of all trades,
master of none).

Felhasználói profil
(pl. kép, név, beállítások)



**DOKUMENTUM
ADATBÁZIS**

Séma nélküli
JSON profilok.

DOKUMENTUMTO
(i. Firestore, MongoDB)

Keresési index
(pl. termékleírások,
keresőszavak)



Szöveges
indexelés.

KERESŐMOTOR
(i. Elasticsearch, Solr)

Kapcsolati háló
(pl. követések,
"ismerősök ismerőse")



Kapcsolatok
követése.

GRÁF ADATBÁZIS
(Neo4j)

Pénzügyi tranzakciók
(ACID, pontosság)



Tranzakciós
integritás.

RELÁCIÓS ADATBÁZIS
(PostgreSQL, MySQL)

JELENTŐSÉG ÉS ELŐNYÖK



TELJESÍTMÉNY OPTIMALIZÁLÁS

Minden adattároló arra a feladatra
van specializálva, amit a legjobban
tud. Gyorsabb válaszidők.



FÜGGETLEN SKÁLÁZHATÓSÁG

Az adattárolók külön
skálázhatók a terhelés alapján.
Csak azt növeljük, ami szükséges.



ADATMODELL FLEXIBILITÁS

Használhatunk sémát vagy
séma nélküli modellt, a fe-
adat jellegének megfelelően.



KÖLTSÉGHATÉKONYSÁG

Specializált és nyílt
forráskódú eszközök
használata, csökkentve
csökkentve a monolitikus
költségeket.

Adattárolási útmutató

- Szöveges adatok (strukturált)
 - adatbázis (pl.: Cloud Firestore)
- Fájlok (képek, zenék, egyéb binárisok)
 - fájl-szintű tároló (pl.: Cloud Storage)
- Felhasználói adatok
 - autentikációs modul
 - saját adatbázisban a jelszavak hash-elve



Optimista vs. pesszimista minta



Gyakorlat

- Bővítsük ki az adattárolást!
 - service-ek mentén hajtsuk végre a CRUD műveleteket
 - implementáljuk a hiányzó üzleti funkciókat is az adatbázis műveletek mentén

